Trabajo Práctico 3: Goleadores del Mundial

Tema: tablas de hash y TAD Diccionario. **Lenguaje:** C++.
Modalidad: en grupos de a 2. **Fecha de entrega:** 5 de noviembre, 23:59.

1 Objetivo

En este trabajo vamos a representar un diccionario de jugadores y goles usando una **tabla de hash**, para que las operaciones de inserción, búsqueda y eliminación sean más rápidas que con un árbol balanceado (ver TAD Diccionario representado sobre un AVL en la teórica con operaciones en $O(\log n)$). Lo vamos a usar para llevar la tabla de goleadores de un Mundial.

El trabajo se organiza en **tres TADs**, cada uno implementado usando únicamente la interfaz del que tiene debajo:

TAD	¿Genérico?	Se representa con
`Goleadores`	No: jugador (`string`) → goles (`int`)	un `Diccionario<string, int>`
`Diccionario<K, V>`	Sí	una `TablaHash<K, V>`
`TablaHash<K, V>`	Sí	un arreglo de *buckets*

Para cada TAD se presenta la **interfaz** (con su contrato y su costo) y la **representación** (con su invariante). El objetivo es realizar la implementación de estos tipos de datos.

Además, `TablaHash` y `Diccionario` ofrecen cada uno un **iterador** para recorrer su contenido (sección 2).

Sobre las complejidades de este enunciado

En todas las cotas, n representa la cantidad de claves guardadas en la estructura en ese momento. Cuando dice **en promedio** se asume la hipótesis de **hashing uniforme** de la teórica: la función de hash reparte las claves de manera uniforme entre los buckets. Cuando dice **amortizado** nos referimos al costo por operación a lo largo de una secuencia de operaciones: alguna puede ser cara (la que redimensiona), pero repartido entre todas el costo es el indicado. Es la misma cuenta de la teórica para InsertarAlFinal del arreglo dinámico y para la redimensión de la tabla de hash. Salvo que se aclare, se supone $T_h \in O(1)$ (con T_h el costo de calcular la función de hash de una clave, como en la teórica) y que comparar dos claves cuesta $O(1)$.
Lo marcado como **optativo** no es necesario para aprobar.

2 TAD Iterador

Es el TAD Iterador de la teórica: abstrae el **recorrido** de una estructura, sin que quien lo usa conozca su representación interna. En este trabajo lo que se recorre son pares $\langle clave, valor \rangle$, así que Actual se separa en dos operaciones.

2.1 Interfaz

- CrearIterador$_{\text{Iterador}}$(*estructura*): crea un iterador sobre *estructura*.
  Pre: — **Post:** devuelve un iterador al primer par de *estructura* (si no tiene pares, el iterador ya está al final).
- HayMas$_{\text{Iterador}}$(it): indica si quedan pares por recorrer.
  Pre: — **Post:** devuelve si quedan pares por recorrer, sin modificar it.
- Clave$_{\text{Iterador}}$(it): la clave del par actual.
  Pre: HayMas$_{\text{Iterador}}$(it). **Post:** devuelve la clave del par actual, sin modificar it.

- VALOR$_{\text{Iterador}}(it)$: el valor del par actual.
  Pre: HAYMAS$_{\text{Iterador}}(it)$. **Post:** devuelve el valor del par actual, sin modificar it.
- AVANZAR$_{\text{Iterador}}(it)$: pasa al par siguiente.
  Pre: HAYMAS$_{\text{Iterador}}(it)$. **Post:** it pasa al par siguiente (o al final, si no hay más).

Un recorrido completo pasa por **cada par exactamente una vez**, en **algún orden** (no se puede asumir cuál). El iterador deja de ser válido si se inserta o se elimina en la estructura mientras se lo usa.

El costo de cada operación depende de la estructura que se recorre, y se indica en la sección de cada una.

2.2 En C++

No hay una única clase `Iterador`: cada estructura define la suya, **anidada** dentro de su propia clase y con estas mismas operaciones. El código que recorre es siempre igual; sólo cambia el tipo del iterador:

```
TablaHash<std::string, int>::Iterador it = tabla.crear_iterador();
while (it.hay_mas()) {
    std::cout << it.clave() << ": " << it.valor() << std::endl;
    it.avanzar();
}
```

En el esqueleto, la declaración de cada `Iterador` ya está escrita: hay que completar su representación (en `TablaHash`) y el cuerpo de sus operaciones.

3 TAD TablaHash

Una tabla de hash con **direccionamiento cerrado**: es un arreglo de *buckets*, donde cada bucket guarda todos los pares $\langle clave, valor\rangle$ cuyo hash cae en esa posición.

Es la misma estructura de la teórica, pero **no la misma interfaz**: allá la tabla era directamente el diccionario (INSERTAR reemplazaba el valor si la clave ya estaba, y PERTENECE y OBTENER eran operaciones de la tabla). Acá la tabla es la capa de abajo, con operaciones más simples y precondiciones más fuertes; el comportamiento de diccionario lo agrega el TAD Diccionario, usándola.

3.1 Interfaz

Con n la cantidad de claves guardadas y c la capacidad (cantidad de buckets) de la tabla:

- TABLAVACIA$_{\text{TablaHash}}(c_0)$: crea una tabla vacía. $O(c_0)$
  Pre: $c_0 \geq 1$. **Post:** devuelve una tabla sin claves, con c_0 buckets.
- INSERTAR$_{\text{TablaHash}}(T, clave, valor)$: agrega un par nuevo. $O(1)$ amortizado, $O(n)$ en peor caso
  Pre: $clave \notin T$. **Post:** T tiene además el par $\langle clave, valor\rangle$.
- ELIMINAR$_{\text{TablaHash}}(T, clave)$: quita $clave$ y su valor. $O(1)$ en promedio, $O(n)$ en peor caso
  Pre: $clave \in T$. **Post:** T ya no tiene a $clave$.
- BUSCAR$_{\text{TablaHash}}(T, clave)$: busca el valor de $clave$. $O(1)$ en promedio, $O(n)$ en peor caso
  Pre: — **Post:** si $clave \in T$, devuelve una referencia a su valor (se puede leer y modificar); si no, NIL.
- CANTIDAD$_{\text{TablaHash}}(T)$: cantidad de claves guardadas. $O(1)$
  Pre: — **Post:** devuelve n.
- CAPACIDAD$_{\text{TablaHash}}(T)$: cantidad de buckets. $O(1)$
  Pre: — **Post:** devuelve c.

- ColisionesTablaHash(T): cantidad de colisiones. $O(c)$ en peor caso
  Pre: — **Post:** devuelve n menos la cantidad de buckets no vacíos (es decir, cuántas claves comparten bucket con otra que llegó antes).
- CrearIteradorTablaHash(T): crea un iterador sobre los pares de T. $O(c)$ en peor caso
  Pre: — **Post:** devuelve un iterador al primer par de T (ver sección 2).

En la implementación, además, la tabla tiene que **liberar toda su memoria** al destruirse (destructor), en $O(n + c)$ en peor caso.

3.2 Iterador de la tabla

Las operaciones son las del TAD Iterador (sección 2), con estos costos:

Operación	Costo
HayMas, Clave, Valor	$O(1)$
Avanzar	$O(c)$ en peor caso
Recorrido completo (CrearIterador y Avanzar hasta el final)	$O(n + c)$ en peor caso

A diferencia del iterador de la lista enlazada, acá Avanzar **no** es $O(1)$: puede tener que pasar por muchos buckets vacíos hasta encontrar el par siguiente. Pero cada bucket y cada par se miran una sola vez en todo el recorrido, y por eso el total es $O(n + c)$.

La representación del iterador también **la eligen ustedes** (¿qué necesita recordar para saber dónde está y cómo seguir?), y también tienen que escribir su invariante como comentario.

3.3 Representación interna (e invariante)

La representación **la eligen ustedes**: en particular, qué estructura usan para cada bucket (una lista enlazada, un arreglo dinámico, ...). Tienen que escribir su **invariante de representación** como comentario en `tabla_hash.h`. Cualquiera sea la representación, tiene que cumplir:

Lo que tiene que valer en cualquier representación

- $c \geq 1$.
- El par de cada clave está en el bucket número $h(clave) =$ `funcion_hash`$(clave)$ mód c.
- Ninguna clave aparece más de una vez.
- $n \leq$ `FACTOR_DE_CARGA_MAXIMO` $\cdot\, c$ (es el α_{max} de la teórica).

El último punto es el que garantiza los costos: como el **factor de carga** $\alpha = n/c$ está acotado por una constante, cada bucket tiene $O(1)$ pares en promedio. Para mantenerlo, cuando una inserción lo rompería hay que **redimensionar**: pasar a un arreglo de buckets más grande (por ejemplo, del doble de capacidad) y reubicar todos los pares (¡el bucket de cada clave cambia, porque cambia c!). No se pide achicar la tabla al eliminar.

En el esqueleto, `FACTOR_DE_CARGA_MAXIMO` $= 2$.

3.4 Función de hash

La tabla llama a `funcion_hash(clave)`, que devuelve un `size_t`. Hay que implementar dos versiones (sobrecarga): una para `std::string`, en $O(m)$ con m la longitud del string, y otra para `int`, en $O(1)$. Lo único obligatorio es que dos claves iguales tengan siempre el mismo hash: **cuál** función usar es parte del trabajo.

3.5 Exploración: ¿qué función de hash conviene?

No hay una única respuesta, así que hay que **probar**. El programa `explorar_hash.cpp` carga en una `TablaHash` los jugadores del archivo de datos del grupo y muestra la cantidad de claves, la capacidad final y las colisiones.

1. Proponer **al menos tres** funciones de hash distintas para strings (alguna deliberadamente mala, por ejemplo una que sólo mire el largo o la primera letra).
2. Para cada una, medir las colisiones sobre el archivo del grupo, con al menos dos capacidades iniciales distintas.
3. Volcar los resultados en una tabla y explicar en pocas líneas qué función eligieron y por qué. ¿Qué tienen en común las claves que colisionan con las funciones malas? ¿Cambia algo si la capacidad es potencia de 2?

Como referencia: si la función repartiera las claves de manera perfectamente uniforme, con n claves y c buckets se esperan aproximadamente $n - c\,(1 - (1 - 1/c)^n)$ colisiones.

4 TAD Diccionario

La misma interfaz de la teórica, más la cantidad de claves. A diferencia de la tabla, acá Insertar **no tiene precondición**: si la clave ya estaba, reemplaza su valor.

4.1 Interfaz

- DiccionarioVacío$_{\text{Diccionario}}()$: crea un diccionario vacío. $O(1)$
  Pre: — **Post:** devuelve el diccionario vacío.
- Insertar$_{\text{Diccionario}}(D, clave, valor)$ $O(1)$ amortizado en promedio, $O(n)$ en peor caso
  Pre: — **Post:** D tiene $clave \to valor$ (reemplazando el valor anterior, si $clave$ ya estaba).
- Pertenece$_{\text{Diccionario}}(D, clave)$: ¿está $clave$? $O(1)$ en promedio, $O(n)$ en peor caso
  Pre: — **Post:** devuelve si $clave$ tiene un valor asociado en D.
- Obtener$_{\text{Diccionario}}(D, clave)$: el valor asociado a $clave$. $O(1)$ en promedio, $O(n)$ en peor caso
  Pre: Pertenece$_{\text{Diccionario}}(D, clave)$. **Post:** devuelve el valor asociado a $clave$.
- Eliminar$_{\text{Diccionario}}(D, clave)$: quita $clave$ (y su valor). $O(1)$ en promedio, $O(n)$ en peor caso
  Pre: Pertenece$_{\text{Diccionario}}(D, clave)$. **Post:** D ya no tiene a $clave$.
- Cantidad$_{\text{Diccionario}}(D)$: cantidad de claves. $O(1)$
  Pre: — **Post:** devuelve n.
- CrearIterador$_{\text{Diccionario}}(D)$: crea un iterador sobre las asociaciones de D. $O(c)$ en peor caso
  Pre: — **Post:** devuelve un iterador a la primera asociación de D (ver sección 2).

En CrearIterador$_{\text{Diccionario}}$, c es la capacidad de la tabla de hash subyacente. Las operaciones del iterador del diccionario tienen los mismos costos que las del iterador de la tabla.

En C++: `bool pertenece(const K& clave)` y `V obtener(const K& clave)`.

4.2 Representación interna (e invariante)

Invariante de representación

D.tabla es una `TablaHash` (que cumple su propio invariante) cuyos pares $\langle clave, valor \rangle$ son exactamente las asociaciones de D.

El diccionario **sólo** puede usar la interfaz pública de `TablaHash`: no conoce los buckets. Presten atención a que cada llamada a la tabla cumpla su precondición.

Lo mismo vale para su iterador, que se representa con un iterador de la tabla:

Invariante de representación del iterador

it.de_la_tabla es un iterador de *D.tabla*, y su par actual es la asociación actual de *it*.

5 TAD Goleadores

La tabla de goleadores de un torneo: para cada jugador que convirtió, cuántos goles lleva. No es genérico.

5.1 Interfaz

Con n la cantidad de jugadores distintos y m la longitud del nombre del jugador (acá las claves son strings, así que $T_h \in O(m)$ y comparar dos claves también cuesta $O(m)$):

- SinGoles$_{\text{Goleadores}}$(): crea una tabla de goleadores vacía. $O(1)$
  Pre: — **Post:** ningún jugador tiene goles.
- AnotarGol$_{\text{Goleadores}}(G, jugador)$: le suma un gol a *jugador*. $O(m)$ amortizado en promedio
  Pre: — **Post:** *jugador* tiene un gol más que antes (si no tenía ninguno, pasa a tener 1).
- Goles$_{\text{Goleadores}}(G, jugador)$: cuántos goles lleva *jugador*. $O(m)$ en promedio
  Pre: — **Post:** devuelve los goles de *jugador* (0 si no anotó ninguno). No modifica G.
- CantidadDeJugadores$_{\text{Goleadores}}(G)$: cuántos jugadores anotaron. $O(1)$
  Pre: — **Post:** devuelve la cantidad de jugadores con al menos un gol.
- MaximoGoleador$_{\text{Goleadores}}(G)$: quién hizo más goles. $O(m)$ en peor caso (no depende de n)
  Pre: CantidadDeJugadores$_{\text{Goleadores}}(G) > 0$. **Post:** devuelve el jugador con más goles; si hay varios empatados, el que llegó primero a esa cantidad.
- **(Optativo)** ImprimirPorNombre$_{\text{Goleadores}}(G)$ $O(n \log n)$ comparaciones, en peor caso
  Pre: — **Post:** imprime una línea "`jugador: goles`" por jugador, en orden alfabético.
- **(Optativo)** ImprimirPorGoles$_{\text{Goleadores}}(G)$ $O(n \log n)$ comparaciones, en peor caso
  Pre: — **Post:** imprime una línea "`jugador: goles`" por jugador, de más a menos goles; a igual cantidad de goles, en orden alfabético.

El orden alfabético es el del operador < entre `std::string`. Los nombres de los archivos de datos no tienen tildes ni eñes.

5.2 Representación interna (e invariante)

$G = \langle G.goles_por_jugador,\ G.maximo \rangle$, donde *G.maximo* es **únicamente un string** (no se guardan aparte sus goles).

Invariante de representación

- *G.goles_por_jugador* es un `Diccionario<string, int>` cuyas claves son exactamente los jugadores con al menos un gol, y el valor de cada uno es su cantidad de goles (siempre ≥ 1).
- Si hay al menos un jugador, *G.maximo* es el nombre de un jugador con la mayor cantidad de goles.

Igual que con *C.maximo* en el Conjunto sobre ABB, el máximo no se recalcula: **cada operación** que modifica G tiene que dejarlo consistente con el diccionario.

Para la parte optativa: una tabla de hash **no guarda las claves en ningún orden útil**, así que para imprimir los listados hay que volcar el contenido del diccionario a un arreglo (recorriéndolo con su iterador) y **ordenarlo**, con un algoritmo de ordenamiento implementado por ustedes.

6 El programa

Cada grupo recibe un archivo de texto con los goles de un Mundial: una línea por gol, con el nombre del jugador que lo hizo. El programa (`main.cpp`) lee esa lista de la **entrada estándar**; la lectura ya está resuelta en el esqueleto, hasta dejar cada nombre en una variable. Falta crear el `Goleadores`, cargarle los goles e imprimir el resumen.

Por ejemplo, para la final de 2022 (`datos/ejemplo.txt`):

Entrada

```
Messi
Di Maria
Mbappe
Mbappe
Messi
Mbappe
```

Salida

```
Jugadores con goles: 3
Maximo goleador: Mbappe (3 goles)
```

Salida adicional de la parte optativa

```
Goleadores por nombre:
Di Maria: 1
Mbappe: 3
Messi: 2

Goleadores por goles:
Mbappe: 3
Messi: 2
Di Maria: 1
```

La salida tiene que respetar **exactamente** ese formato. Se corre así:

```
./goleadores < datos/ejemplo.txt
```

7 Archivos y reglas

Archivo	Qué hay que hacer
`funcion_hash.h` / `.cpp`	Implementar las dos funciones de hash.
`tabla_hash.h`	Elegir la representación e implementar `TablaHash<K, V>` y su iterador.
`diccionario.h`	Implementar las operaciones de `Diccionario<K, V>` y de su iterador.
`goleadores.h` / `.cpp`	Implementar las operaciones de `Goleadores`.
`main.cpp`	Completar el programa (la lectura ya está).
`explorar_hash.cpp`	Completar el programa de exploración (la lectura ya está).
`tests.cpp`	Pruebas de la parte obligatoria. Agreguen las suyas.
`Makefile`	`make`, `make test`, `make ejemplo`, `make valgrind`; `make ejemplo-optativo` (los listados de la parte optativa).

`TablaHash` y `Diccionario` son clases template, como la lista enlazada del laboratorio: su implementación va en el mismo `.h`. Las operaciones de cada `Iterador` se completan directamente dentro de la clase anidada. Lo único que se puede hacer con una clave es copiarla, compararla con == y pasársela a `funcion_hash`; con un valor, copiarlo.

Reglas

- **No se puede cambiar la interfaz pública** de ninguna de las clases. La parte privada de `TablaHash` y de su iterador es libre; la de `Diccionario`, su iterador y `Goleadores` es la de este enunciado (se pueden agregar funciones auxiliares privadas).
- De la biblioteca estándar sólo se puede usar `<string>`, `<iostream>`, `<cstddef>` y `<vector>`. En particular, **no** se puede usar `std::map`, `std::unordered_map`, `std::hash` ni `std::sort`.
- Cada TAD usa sólo la interfaz pública del de abajo.
- El programa no puede perder memoria ni acceder a memoria inválida (`make valgrind` tiene que terminar sin errores).
- Tienen que cumplirse las complejidades de este enunciado.

8 Entrega

Se entrega por el campus, hasta el 5 de noviembre a las 23:59, un único archivo comprimido por grupo con:

- Todos los archivos del esqueleto, completos.
- El archivo de datos que le tocó al grupo, como `datos/goles.txt`.
- La salida del programa para ese archivo, como `datos/salida.txt` (`./goleadores < datos/goles.txt > datos/salida.txt`).
- La exploración de funciones de hash, como `exploracion.txt`: la tabla de resultados y la explicación.

Para aprobar, el trabajo tiene que compilar sin errores, pasar las pruebas de la cátedra (que incluyen a las de `tests.cpp`), no tener errores de memoria, respetar las reglas de la sección anterior e incluir la exploración.